# Module 10 Lab Session 2: The Identity Handshake

| | |
|---|---|
| **Module** | M10 Full-Stack Integration |
| **Session** | 2 of 3 |
| **Exercises** | **Exercise 2:** HttpOnly auth cookie, XSRF double-submit protection, credentials interceptor, AuthService |

## Story Thread

Abeba, a senior TMS administrator, logs in to review student enrollment records. In many basic tutorials, developers save JWT access tokens inside browser `localStorage`.

Why is that a major security hazard in enterprise applications?

If an attacker manages to inject a Cross-Site Scripting (XSS) payload through a forum post, profile bio, or compromised npm dependency, a single line of malicious JavaScript `localStorage.getItem('auth_token')` can exfiltrate Abeba's admin token to an external server. With that token, the attacker now owns the system.

In this lab, we build **The Identity Handshake**:

1. **HttpOnly Authentication Cookies:** The .NET API writes the auth token into an `HttpOnly` cookie. Because it is marked `HttpOnly`, client-side JavaScript (including malicious XSS scripts) is physically incapable of reading it. The browser attaches it automatically to outgoing requests.

2. **XSRF Double-Submit Protection:** Because browsers automatically send cookies on cross-site requests, we must guard against Cross-Site Request Forgery (CSRF). We issue a secondary, readable cookie (`XSRF-TOKEN`). Angular reads this cookie and echoes it back in an `X-XSRF-TOKEN` HTTP header on mutating requests (POST, PUT, DELETE). Since malicious external sites cannot read cookies across origins under SOP rules, they cannot supply the matching header.

3. **Client Session State Wrapper (`AuthService`):** Angular's `AuthService` drives the login sequence and tracks current user session signals without exposing raw tokens to DOM scripts.

## Exercise 2: The Identity Handshake (HttpOnly Cookie + XSRF)

### Part A: Server-Side Cookie Issuance in .NET

First, let's create (or update) `Controllers/AuthController.cs` in our Web API project to issue secure `HttpOnly` cookies upon login instead of returning raw tokens in the response body.

> **Note:** In Module 10, we build the **browser cookie transport layer**. The full database-backed ASP.NET Core Identity system, BCrypt password hashing, and JWT signing keys will be built in **Module 12 (Security & Authentication)**.

1. Create Auth DTOs in Application layer `LoginRequest.cs and UserProfile.cs`

```
public record LoginRequest(string Username, string Password);
public record UserProfileDto(string DisplayName, string Role);
```

2. Create a new file `Controllers/AuthController.cs` in your Web API project:

```
namespace TmsApi.Api.Controllers;

[ApiController]
[Route("api/{version:apiVersion}/auth")]
public class AuthController : ControllerBase
{
    [HttpPost("login")]
    public IActionResult Login(
        [FromBody] LoginRequest request,
        [FromServices] IWebHostEnvironment env)
    {
        // Validate credentials (demo account for M10 transport
testing)
        if (request.Username == "admin" && request.Password ==
"Password123!")
        {
            var dummyJwt = "header.payload.signature-demo-token";

            // Append HttpOnly authentication cookie — JavaScript
CANNOT read this token
            Response.Cookies.Append("tms_auth", dummyJwt, new
CookieOptions
            {
                HttpOnly = true,
                Secure = !env.IsDevelopment(), // HTTPS in prod;
HTTP permitted locally over dev
                SameSite = SameSiteMode.Strict,
```

```
                Expires = DateTimeOffset.UtcNow.AddHours(2)
            });

            return Ok(new UserProfileDto("System Admin",
"Admin"));
        }

        return Unauthorized(new { detail = "Invalid username or
password." });
    }

    [HttpGet("me")]
    public IActionResult GetCurrentUser()
    {
        // Inspect cookie attached automatically by the browser
on cross-origin requests
        if (Request.Cookies.TryGetValue("tms_auth", out _))
        {
            return Ok(new UserProfileDto("System Admin",
"Admin"));
        }

        return Unauthorized(new { detail = "Session expired or
missing authentication cookie." });
    }
}
```

Notice `Secure = !env.IsDevelopment()`. In local development over HTTP (`http://localhost:5000`), setting `Secure = true` will cause browser security policies to reject the cookie immediately.

## Part B: Configure Antiforgery Middleware in .NET

Next, let's configure ASP.NET Core to generate anti-forgery tokens for state-changing operations.

1. Open `Program.cs` in your Web API project.
2. Register the Antiforgery service with a header name matching Angular's default convention:

```
using Microsoft.AspNetCore.Antiforgery;

builder.Services.AddAntiforgery(options =>
{
    options.HeaderName = "X-XSRF-TOKEN";
});
```

3. Add middleware to issue the readable `XSRF-TOKEN` cookie whenever an authenticated user communicates with the API. Place this middleware *after* `app.UseAuthentication()` and `app.UseAuthorization()`:

```
app.Use(async (context, next) =>
{
    if (context.User.Identity?.IsAuthenticated == true || context.
Request.Cookies.ContainsKey("tms_auth"))
    {
        var antiforgery = context.RequestServices
            .GetRequiredService<IAntiforgery>();
        var tokens = antiforgery.GetAndStoreTokens(context);

        context.Response.Cookies.Append("XSRF-TOKEN", tokens.Requ
estToken!,
            new CookieOptions
            {
                HttpOnly = false, // MUST be false so Angular Jav
aScript can read it!
                Secure = !builder.Environment.IsDevelopment(),
                SameSite = SameSiteMode.Strict
            });
    }
    await next(context);
});
```

## Part C: Configure Angular Credentials Interceptor and XSRF Handshake

By default, Angular's `HttpClient` omits cookies when making HTTP requests across origins. We will write an HttpInterceptor to attach credentials automatically, and configure Angular's built-in XSRF protection.

1. Create a file named `src/app/interceptors/credentials.interceptor.ts`:

```
import { HttpInterceptorFn } from '@angular/common/http';

export const credentialsInterceptor: HttpInterceptorFn = (req, ne
xt) => {
  return next(req.clone({ withCredentials: true }));
};
```

2. Open `src/app/app.config.ts` and register the interceptor along with `withXsrfConfiguration`:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([credentialsInterceptor]),
      withXsrfConfiguration({
        cookieName: 'XSRF-TOKEN',   // Cookie name set by .NET
server
        headerName: 'X-XSRF-TOKEN', // Header expected by .NET
server
      })
    )
  ]
};
```

Now, every HTTP request emitted by `HttpClient` automatically includes `withCredentials: true`. Furthermore, Angular detects the `XSRF-TOKEN` cookie and automatically adds the `X-XSRF-TOKEN` header to all POST, PUT, and DELETE requests!

## Part D: Build `AuthService` for Cookie-Backed Sessions

Now let's build `AuthService` to manage current user session state in Angular using Signals.

1. Open `src/app/services/auth.service.ts` and implement cookie-backed authentication state management:

   ```
   export interface TmsUser {
     displayName: string;
     role: string;
   }

   export interface LoginRequest {
     username: string;
     password: string;
   }

   @Service()
   export class AuthService {
     private http = inject(HttpClient);
     currentUser = signal<TmsUser | null>(null);

     hasRole(role: string): boolean {
       const user = this.currentUser();
       return user?.role === role || user?.role === 'Admin';
     }
   ```

```
  async login(credentials: LoginRequest) {
    // Server sets the HttpOnly cookie in the Set-Cookie response
header
    await firstValueFrom(
      this.http.post<void>('/api/auth/login', credentials)
    );

    // Fetch authenticated profile — browser automatically sends
the cookie
    const user = await firstValueFrom(
      this.http.get<TmsUser>('/api/auth/me')
    );
    this.currentUser.set(user);
  }
}
```

**Transport vs. Role Authorization:** `AuthService` tracks the current session state on the client. In **Module 12 (Security & Authentication)**, you will pair this service with production functional route guards (`roleGuard`) and ASP.NET Core Identity claims policies.

## Verifying Your Work

Launch your app, log in as an administrator or student, and inspect the state in Chrome DevTools:

1. **Inspect Cookies:** Open DevTools -> **Application** tab -> **Cookies** (`http://localhost:4200` or `localhost:5000`).
    - Find `tms_auth`: Check that the **HttpOnly** column has a checkmark.
    - Find `XSRF-TOKEN`: Check that the **HttpOnly** column is **blank** (allowing Angular JavaScript access).
2. **Verify Storage is Clean:** Click **Local Storage** in DevTools. Ensure no tokens or sensitive session data remain stored there.
3. **Verify Header Insertion:** In the DevTools **Network** tab, trigger a POST action (e.g., submitting an enrollment). Inspect the Request Headers you will see `X-XSRF-TOKEN` populated with the value matching your `XSRF-TOKEN` cookie!

## What's Next?

You now have a secure, cookie-backed identity handshake operating cleanly between Angular and .NET.